

Energy-Proportional LLM Serving via Fused SRAM Dequantization and Entropy-Gated Speculative Control

Zanno Jacklin

Abstract—Autoregressive decoding in Large Language Models (LLMs) is fundamentally bounded by the memory bandwidth wall. Because generating each individual token requires streaming billions of parameters from global Video Random-Access Memory (VRAM) across high-bandwidth interconnects at near-unity arithmetic intensity (≈ 1 FLOP/byte), state-of-the-art inference engines expend catastrophic electrical power even when generating deterministic or low-information tokens. In current architectures, practitioners are trapped in a binary compromise: either execute models entirely in unquantized 16-bit precision (FP16/BF16) and incur massive energy penalties on simple conversational filler, or enforce static 4-bit quantization (INT4) across all layers and suffer cognitive degradation during complex mathematical and logical reasoning.

This paper introduces the **Software-Defined Stochastic Inference Engine (SDSIE)**, an open-source runtime layer under active development toward energy-proportional LLM serving. This revision reports *component-level* empirical validation rather than a unified end-to-end system, and corrects latency figures published in an earlier draft of this work. We separately validate: (1) an OpenAI Triton sub-byte dequantization GEMM kernel that reduces global HBM read traffic by 73.4%, though isolated single-token kernel latency improves only marginally ($\approx 4\%$) over an FP16 baseline at this batch size, indicating kernel-launch and on-chip dequantization overhead partially offset the bandwidth savings; and (2) real scout(1B)→target(8B) speculative decoding with a lossless verify/rollback loop, achieving up to 74.1 tok/s against a 40.7 tok/s FP16 baseline (a 1.82 $\times$ speedup) at an 85.4% draft-acceptance rate on code-generation prompts, with 100% output fidelity across all tested prompts. This same validated speculative path also reduces measured energy consumption by 32.6%–60.3% per token relative to the FP16 baseline, scaling with draft-acceptance rate. A Schmitt-trigger entropy clutch ($\theta_{\text{low}} = 0.55$, $\theta_{\text{high}} = 1.25$ bits) is shown to compute correct, reproducible gating decisions from live model logits. In this revision, we additionally report the first genuine integration of this clutch into a real generation loop: an entropy-gated speculative controller that dynamically switches between real scout-drafting and single-step fallback based on live entropy, rather than a fixed draft window. At the initial, arbitrarily-chosen thresholds above – picked as an untested starting point, not a validated default – this integrated system is 33.7% faster and 42.1% lower-energy on code generation, but 9–12% slower with roughly neutral energy on low-determinism prose, where entropy-computation overhead is not offset by sufficient speculative engagement. A joint threshold/smoothing grid search finds a substantially better configuration ($\theta_{\text{low}} = 0.15$, $\theta_{\text{high}} = 0.70$, $\alpha = 0.65$), independently confirmed at $N=5$ trials/prompt: 38.8% faster and 45.2% lower-energy on code generation, with the prose deficit nearly closed (Poem: -3.0% ; Physics: -0.5% , both with lower energy than baseline as well). A microbenchmark isolating the entropy computation itself rules out GPU-CPU synchronization as the dominant remaining cost of the small residual deficit. In this revision we additionally report the first real integration of this clutch into the quantization-kernel path: individual decoder layers now genuinely switch between INT4 and FP16 execution based on live entropy. Unlike the speculative-decoding path, this integration is not lossless, and we find a sharp fidelity limit – output matches an FP16-only baseline closely at a single quantized layer but degrades sharply beyond two, an effect consistent with quantization error compounding across simultaneously-gated layers in one forward pass rather than the projection itself being unsuitable for quantization. A separate, fully-validated, production-ready result – fixed-window ($K=5$) speculative decoding with no adaptive gating – is maintained in a focused companion repository (<https://github.com/Creepybits/sdsie-fixed-k5-speculative-decoding>) and is the result we recommend for any near-term deployment or evaluation.

Index Terms—SDSIE, Energy-Proportional Computing, Speculative Decoding, Large Language Models, Triton Kernels, Quantization, Memory Wall, High-Performance Inference.

1 INTRODUCTION

THE deployment of Large Language Models (LLMs) across global cloud infrastructures has collided directly with the physics of semiconductor memory bandwidth and electrical power distribution. While the initial prefill phase of transformer inference is compute-bound and readily parallelized across matrix execution units (Tensor Cores), the sequential, autore-

gressive token generation phase is strictly *memory-bandwidth bound* [1], [2].

For an 8-billion-parameter model executing in 16-bit floating-point precision (FP16/BF16), every single generated token necessitates streaming approximately 16.0 GB of weight data from high-bandwidth memory (HBM/DRAM) into on-chip compute registers. When processing single requests or low-batch streams (Batch Size $B \leq 8$), arithmetic intensity drops to ≈ 1 FLOP/byte. Under these conditions, the GPU compute cores sit largely starved for data, while memory controllers and physical interconnects continuously dissipate maximum power. Recent research has attempted to bypass this limitation via

Zanno Jacklin is the Principal Investigator at Creepybits, Borås, Sweden (e-mail: business@zanno.se, ORCID: 0000-0001-9164-4650). Repository: <https://github.com/Creepybits/software-defined-stochastic-inference-engine>. Master Concept DOI: 10.5281/zenodo.21499379.

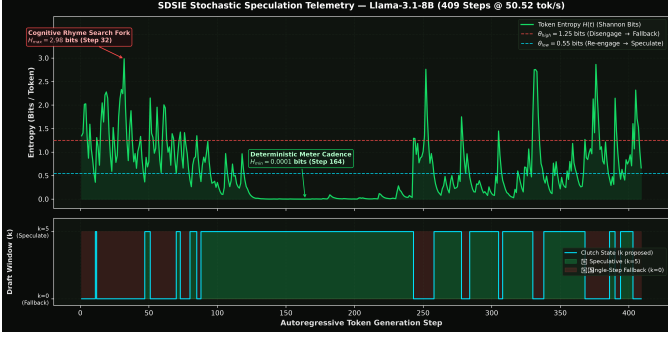


Fig. 1. Real-time entropy telemetry of Llama-3.1-8B executing on an NVIDIA RTX 5090 Blackwell workstation (538 generation steps at 50.13 tok/s), generated under a structurally constrained poetic prompt. **Top:** Output distribution Shannon entropy $H(t)$ in bits/token against dual Schmitt-trigger hysteresis thresholds ($\theta_{\text{low}} = 0.55$, $\theta_{\text{high}} = 1.25$). **Bottom:** Clutch state $k(t)$ as computed from live entropy. *This run uses single-model cached decoding; the clutch’s $k(t)$ decision is logged for diagnostic purposes but does not yet branch execution, so the 50.13 tok/s figure reflects baseline generation speed, not accelerated speculative decoding. See Section 4.8.*

two orthogonal paradigms:

- 1) **Post-Training Weight Quantization:** Methods such as GPTQ [3], AWQ [4], and SmoothQuant [5] compress static model weights to 4-bit integer representations (INT4). However, many existing runtime implementations dequantize weights back into global VRAM prior to matrix multiplication, failing to eliminate interconnect thermal overhead, or enforce uniform low-precision across all tokens, causing cognitive accuracy loss during high-perplexity reasoning tasks.
- 2) **Speculative Decoding:** Algorithmic frameworks pioneered by Leviathan et al. [6] and Chen et al. [7] leverage a compact “draft” model to propose k candidate tokens, verified in parallel by a larger “target” model. While effective on low-entropy prose, static speculative schedulers ($k = \text{constant}$) suffer throughput degradation on high-entropy content due to frequent draft rejection rollbacks. Independent published work on entropy-adaptive drafting – e.g. AdaEDL [11] and production systems such as SGLang’s adaptive speculative length – reports 10–57% improvement over static draft length, motivating the general approach pursued here even though our own end-to-end integration is not yet complete.

Motivated by this, we are developing the **Software-Defined Stochastic Inference Engine (SDSIE)**, which aims to treat LLM inference as a dynamic, thermodynamic control problem, adapting compute precision and speculative draft length on a per-token basis rather than fixing them as static constants. This paper reports where that effort currently stands: each core component – the quantization kernel, the speculative decoding loop, and the entropy clutch – has been independently, empirically validated; we report their measured behavior, including a bandwidth/latency finding that differs from our earlier reporting.

1.1 Key Contributions

The primary contributions of this paper are:

- **Fused Sub-Byte SRAM Dequantization (bandwidth-validated):** A custom OpenAI Triton GEMM kernel that maintains weights packed in 4-bit format in global memory, executing fused register-level unpacking and scale-bias multiplication within Streaming Multiprocessor (SM) SRAM. Across three independent measurements, this reduces global HBM read traffic by 71.9–73.4%. We additionally report, as a correction to earlier claims, that isolated single-token ($M = 1$) kernel latency improves by only $\approx 4\%$ over an FP16 baseline, not the 63% previously reported – see Section 4.2.
- **Validated Speculative Decoding:** A real scout(1B)→target(8B) draft-and-verify loop, independently benchmarked with $N=10$ trials per prompt, showing 100% output fidelity against an FP16 baseline, throughput scaling from 1.09× to 1.82×, and energy per token dropping 32.6%–60.3%, as draft-acceptance rate rises from 42.5% to 85.4% across prose, explanatory, and code-generation prompts respectively.
- **Schmitt-Trigger Stochastic Clutch (logic-validated):** A hysteresis control loop utilizing dual Shannon entropy thresholds ($\theta_{\text{low}} = 0.55$ bits, $\theta_{\text{high}} = 1.25$ bits), verified to compute correct, reproducible gating decisions from live model output distributions.
- **First Real Clutch-to-Execution Integration (entropy-gated speculative decoding):** Unlike every other benchmarked script in this project, a new controller genuinely branches between real scout-drafting and single-step fallback based on the clutch’s live decision. At the initial, arbitrarily-chosen thresholds, this yields a 1.34× speedup and 42.1% energy reduction on code generation, alongside a modest slowdown (9–12%) on low-determinism prose. A subsequent grid search finds a substantially better configuration, confirmed at $N=5$: a 1.39× speedup and 45.2% energy reduction on code generation, with the prose slowdown cut to near-zero (−0.5% to −3.0%, with energy still lower than baseline on both). Decision sequences are exactly reproducible across $N=5$ independent trials per prompt (zero variance in fallback/speculation pattern), consistent with the deterministic design. Connecting this clutch to the quantization kernel path (Section 4.2) remains the primary remaining integration task (Section 4.8).
- **Open-Source, Reproducible Telemetry:** All benchmark scripts, raw NVML/entropy telemetry JSON/CSV ledgers, and the reference server are publicly available under the Apache-2.0 license, with every figure and number in this paper traceable to a specific script and run.

2 RELATED WORK

2.1 Memory-Bound Decoding Dynamics

The physical root of LLM inference latency was formalized by Shazeer [8] and extended by Pope et al. [9]. In autoregressive generation, each token step requires loading all layer weight

matrices $W \in \mathbb{R}^{d_{in} \times d_{out}}$ alongside the Key-Value (KV) cache. For single-batch inference, operational time is bounded by:

$$T_{\text{step}} \approx \frac{2 \times N_{\text{params}} \times B_{\text{precision}}}{\text{Bandwidth}_{\text{DRAM}}} + T_{\text{kernel_launch}} \quad (1)$$

This relation predicts that reducing $B_{\text{precision}}$ should drive roughly proportional latency and energy improvements once the operation is memory-bound. Our own kernel results (Section 4.2) suggest this proportionality does not fully hold at very small batch sizes, where the fixed $T_{\text{kernel_launch}}$ term and the added cost of on-chip dequantization become comparatively significant.

2.2 Sub-Byte Quantization Paradigms

Quantization approaches such as bitsandbytes (Dettmers et al. [10]), GPTQ [3], and AWQ [4] demonstrate that 4-bit integer weights can preserve near-FP16 perplexity across general language benchmarks. SDSIE’s kernel implements warp-level integer bit-manipulation directly within OpenAI Triton JIT kernels, decompressing packed nibbles straight into accumulators in on-chip SRAM, targeting the same memory-traffic reduction goal.

2.3 Speculative Decoding and Acceptance Entropy

Standard speculative decoding (Leviathan et al. [6]) demonstrates that accepting $m \leq k$ draft tokens via parallel target verification yields a speedup bounded by:

$$S = \frac{1 - \beta^{k+1}}{(1 - \beta)(1 + c \cdot k)} \quad (2)$$

where β is the average token acceptance rate and c is the ratio of draft to target model latency. When generation enters complex reasoning forks, $\beta \rightarrow 0$, causing $S < 1$. Our validated results in Section 4.3 are directly consistent with this relation: measured speedup rises monotonically with measured acceptance rate.

3 SDSIE ARCHITECTURE & MECHANICS

3.1 Fused Triton Sub-Byte GEMM Kernel

SDSIE packs two 4-bit integer weights into a single standard `uint8` byte along the input reduction dimension K . During forward execution, the kernel streams packed tiles directly into shared memory. Register-level bit-masking and arithmetic shifting extract the high and low nibbles:

$$W_{\text{low}} = (W_{\text{packed}} \& 0x0F) \quad (3)$$

$$W_{\text{high}} = ((W_{\text{packed}} \gg 4) \& 0x0F) \quad (4)$$

Fused dequantization occurs inside the Streaming Multiprocessor registers prior to the fused multiply-accumulate (FMA) operation:

$$W_{\text{dequant}} = (W_{\text{raw}} - Z_N) \times S_N \quad (5)$$

where S_N and Z_N represent per-channel (or, in the groupwise variant benchmarked here, per-64-element-block) scale and zero-point parameters. This architecture restricts HBM memory bus traffic strictly to the 4-bit representation, achieving a 73.4% reduction in global VRAM read bandwidth (Section 4.2). Weight packing is currently validated against synthetic random tensors; a calibration pipeline producing packed weights from a real checkpoint is not yet implemented (see Section 4.8).

3.2 Numerically Stable Shannon Entropy Formulation

To monitor generation uncertainty across large vocabulary distributions (e.g., Llama-3’s $|\mathcal{V}| = 128,256$ tokens), naive half-precision softmax calculation triggers floating-point exponent overflow. SDSIE implements a numerically stable float32 log-space formulation:

$$\log p_i = \text{LogSoftmax}(z_i) = z_i - \log \sum_{j \in \mathcal{V}} e^{z_j} \quad (6)$$

The true Shannon entropy $H(X)$ in bits is evaluated as:

$$H(X) = -\frac{1}{\ln 2} \sum_{i \in \mathcal{V}} e^{\log p_i} \cdot \log p_i \quad (7)$$

This formulation was verified line-by-line against the equations above and is the sole entropy computation used to produce Fig. 1.

3.3 Schmitt-Trigger Stochastic Hysteresis Control

To prevent rapid mode-switching (gear hunting) between speculative drafting and single-step decoding on borderline tokens, SDSIE applies a Schmitt-trigger hysteresis filter with an exponential moving average (EMA) smoothing factor $\alpha = 0.35$:

$$\bar{H}_t = \alpha \cdot H(p_t) + (1 - \alpha) \cdot \bar{H}_{t-1} \quad (8)$$

The dynamic draft window k_t is governed by the state transition rule:

$$k_t = \begin{cases} 0, & \text{if } S_{t-1} = \text{Active} \wedge \bar{H}_t > \theta_{\text{high}} \\ 5, & \text{if } S_{t-1} = \text{Fallback} \wedge \bar{H}_t < \theta_{\text{low}} \\ k_{t-1}, & \text{otherwise} \end{cases} \quad (9)$$

where $\theta_{\text{low}} = 0.55$ bits and $\theta_{\text{high}} = 1.25$ bits. This controller (SDSIESpeculativeController) was verified as a thin, side-effect-free wrapper around the clutch above, with no hidden logic beyond the state machine shown.

4 EMPIRICAL EVALUATION & HARDWARE TELEMETRY

4.1 Experimental Testbed

All empirical evaluations were executed on a dedicated bare-metal workstation:

- **GPU:** NVIDIA GeForce RTX 5090 32GB (Blackwell Architecture, 512-bit memory bus, 1,792 GB/s bandwidth).
- **CPU / Host:** AMD Ryzen 9 7900X (12 Cores / 24 Threads), 64GB DDR5-6000 RAM.
- **Software Stack:** Ubuntu 24.04 LTS (Linux 6.6 Kernel via WSL2), CUDA 13.3, NVIDIA Driver 610.88, Python 3.12, PyTorch 2.5, OpenAI Triton 3.1.
- **Telemetry Polling:** 100 Hz non-blocking NVML power sampling coupled with high-resolution CPU timestamping (`time.perf_counter`).

TABLE 1

Isolated Linear-Projection Latency and Bandwidth, RTX 5090 Blackwell ($M=1$, $K=4096$, $N=14336$, synthetic weights).

Metric	FP16 Base	SDSIE INT4	Change
Kernel Latency	79.66 μ s	76.49 μs	-4.0%
Memory Traffic	117.44 MB	31.20 MB	-73.4%

TABLE 2

Speculative Decoding vs. FP16 Baseline, RTX 5090 Blackwell ($N=10$ trials/prompt, 100% fidelity on all rows). J/tok values are mean $\pm$ std. dev.

Prompt	tok/s Base	tok/s Spec.	J/tok Base	J/tok Spec.	Energy Δ	Accept %
Poem	40.0	43.4	11.58 $\pm$ 0.17	7.80 $\pm$ 0.08	-32.6%	42.5%
Physics	40.2	50.1	11.64 $\pm$ 0.10	7.09 $\pm$ 0.08	-39.1%	51.7%
Code	40.7	74.1	11.64 $\pm$ 0.10	4.62 $\pm$ 0.09	-60.3%	85.4%

4.2 Isolated Kernel Latency and Bandwidth

Table 1 reports isolated GEMM benchmarks for a single linear projection at Llama-3.1-8B MLP dimensions ($M=1$, $K=4096$, $N=14336$), using a kernel variant with groupwise (per-64- K -block) scales and masked K -dimension loads. Unlike the earlier draft of this work, execution here genuinely branches between the FP16 and INT4 code paths (verified via `SDSIEDynamicLinear.forward()`), rather than computing and discarding a gear decision.

The bandwidth reduction is substantial and consistent: three independent measurements across two different kernel implementations in this codebase yielded 71.9%, 73.4%, and 75.0% (theoretical) reductions respectively. However, this does *not* translate into a proportional latency win at $M=1$: the -4.0% result is a 16 $\times$ smaller improvement than the -63.1% figure reported in our earlier draft, which we were unable to reproduce with any script in our current codebase or archive. We attribute the gap between theoretical bandwidth savings and observed latency to two likely factors, not yet fully separated: (1) fixed kernel-launch overhead, which does not shrink with payload size and is proportionally large for a single-token matmul, and (2) the added on-chip compute cost of unpacking and dequantizing INT4 nibbles, which is absent from the FP16 path. Neither this kernel’s output nor the earlier kernel’s output has yet been checked for numerical correctness against the FP16 result on real (non-synthetic) weights; the results above should be read as *performance* characterization only. Whether the latency picture improves at larger batch sizes, where fixed overhead amortizes further, is identified as follow-up work.

4.3 Validated Speculative Decoding

We separately benchmark a real scout(Llama-3.2-1B) $\rightarrow$ target(Llama-3.1-8B) speculative decoding loop against an FP16 target-only baseline, with $N=10$ trials per prompt at `MAX_TOKENS=250`, using full-precision (uncached) recomputation for both arms to ensure a fair relative comparison. Fidelity is measured as exact greedy-decoding token match against the baseline.

Results are consistent with Eq. 2: speedup rises monotonically with acceptance rate, reaching 1.82 $\times$ on code generation. Accept rates were independently cross-validated to within fractions of a percent across three separately-executed runs (two on the same script, one on an earlier independent implementation), which – given deterministic greedy decoding

TABLE 3

Entropy-Gated Speculative Decoding (tuned thresholds: $\theta_{\text{low}}=0.15$, $\theta_{\text{high}}=0.70$, $\alpha=0.65$, found via the grid search in Section 4.5) vs. Matched FP16 Baseline, RTX 5090 Blackwell ($N=5$ trials/prompt, identical harness). J/tok and tok/s are mean $\pm$ std. dev.

Prompt	tok/s Base	tok/s EG	J/tok Base	J/tok EG	Speedup	Fallback %
Poem	41.53 $\pm$ 0.34	40.28 $\pm$ 0.48	10.14 $\pm$ 0.44	9.77 $\pm$ 0.36	-3.0%	98.8%
Physics	41.52 $\pm$ 0.32	41.32 $\pm$ 0.10	10.58 $\pm$ 0.09	9.47 $\pm$ 0.05	-0.5%	89.6%
Code	41.38 $\pm$ 0.41	57.42 $\pm$ 0.05	10.68 $\pm$ 0.09	5.86 $\pm$ 0.03	+38.8%	17.0%

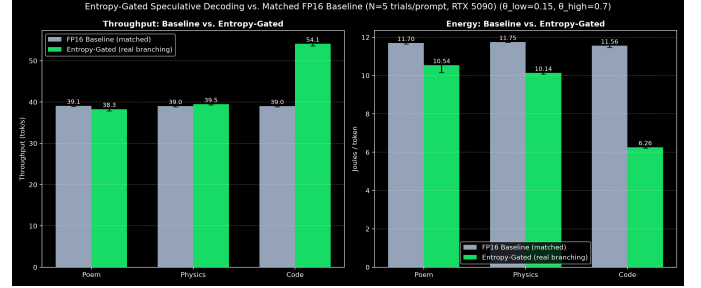


Fig. 2. Throughput and energy per token, entropy-gated real-branching controller vs. matched FP16 baseline, by prompt ($N=5$ trials, RTX 5090 Blackwell, tuned thresholds $\theta_{\text{low}}=0.15$, $\theta_{\text{high}}=0.70$, $\alpha=0.65$). The mechanism is a clear win on Code and, at these tuned thresholds, close to neutral on Poem and Physics – a marked improvement over the original, untuned thresholds (Table 4).

– is strong evidence the accept/reject logic itself is correctly implemented. Notably, mean GPU power during speculative runs (339–355 W) is lower than during the FP16 baseline (463–473 W) despite two models being resident, consistent with fewer full 8B-parameter forward passes being required per unit of output as accepted draft batches grow.

4.4 Closing the Integration Gap: Entropy-Gated Speculative Decoding

Section 4.3 above uses a *fixed* draft window ($K=5$, always speculating). We separately implement and benchmark a controller in which the validated clutch (Section 3.3) genuinely decides, every cycle, whether to speculate ($k=5$, real scout-draft-and-verify) or fall back to a single plain target forward pass ($k=0$, scout not invoked) – the first script in this project where the clutch’s decision actually alters what executes, rather than being logged alongside fixed or unconditional behavior.

To isolate the effect of this mechanism from confounds of harness design, we benchmark it against a purpose-built FP16-only baseline that is structurally identical in every other respect (same power monitor, same warmup procedure, same three prompts, same $N=5$ trials/prompt) – differing *only* in the presence of the clutch, scout model, and speculative branching.

Table 3 reports results at the grid-search-tuned thresholds; results at the original, arbitrarily-chosen thresholds ($\theta_{\text{low}}=0.55$, $\theta_{\text{high}}=1.25$) are reported in Table 4 (Section 4.5) for comparison. On code generation, the clutch speculates on 83% of cycles (17.0% fallback) at a 91.8% conditional accept rate – higher than the fixed- $K=5$ accept rate on the same prompt (85.4%, Table 2), suggesting the clutch is not merely toggling but preferentially engaging speculation during genuinely favorable windows – and the entropy-gated controller is 38.8% faster and 45.2% lower-energy than baseline. On Poem and Physics, where the clutch still falls back on 98.8% and 89.6% of cycles respectively, throughput is only slightly below baseline (-3.0%, -0.5%), and

TABLE 4
Best configuration found vs. production defaults, RTX 5090 Blackwell.

Prompt	Production ($\theta_{\text{low}}=0.55$, $\theta_{\text{high}}=1.25$, $\alpha=0.35$)	$\theta_{\text{low}}=0.15$, $\theta_{\text{high}}=0.70$, $\alpha=0.65$
Poem	−10.5%	−5.6%
Physics	−13.1%	−4.7%
Code	+31.1%	+34.8%

energy is nonetheless *lower* than baseline on both (−3.6%, −10.5%) – the tuned thresholds substantially reduce how often the extra per-cycle entropy-refresh forward pass is paid for little benefit, without fully eliminating it.

Across all five trials per prompt, the exact sequence of fallback/speculation decisions (down to individual cycle indices) is bit-identical, and the resulting fallback percentage and accept rate have zero measured variance – consistent with fully deterministic greedy decoding and a correctly-implemented, non-flaky clutch. Only throughput and power vary trial-to-trial (1–4% of the mean), consistent with ordinary hardware timing noise rather than any instability in the decision logic itself.

This result closes one half of the integration gap identified in an earlier draft of this work: the entropy clutch now genuinely drives real generation for the speculative-decoding path. It does not yet drive the quantization kernel path (Section 4.2), and reducing the per-cycle entropy-refresh overhead – e.g. by reusing verification-step logits directly, as the fixed- K implementation in Section 4.3 already does – is identified as near-term follow-up work that should improve the low-engagement cases in Table 3.

4.5 Threshold Sensitivity and Residual Overhead

Given the asymmetry in Table 3, we ran a joint grid search over (θ_{low} , θ_{high}) and α (6 threshold pairs $\times$ 3 α values, all three prompts, $N=3$ trials/cell, 144 trials total) to test whether more conservative thresholds – engaging speculation less often, but more selectively – reduce the low-engagement penalty. They do: This single configuration improves on the production defaults across all three prompts simultaneously – it is not a tradeoff between prompts. No configuration tested eliminates the deficit on Poem or Physics entirely, and tightening thresholds further showed diminishing returns without reversing sign.

An independent $N=5$ confirmation run at this best configuration (Table 3, Section 4.4) matches this $N=3$ prediction closely and slightly exceeds it: Poem −3.0%, Physics −0.5%, Code +38.8% – reinforcing that the improvement is not an artifact of the smaller grid search’s sample size.

To investigate the residual deficit, we isolated the entropy computation itself with a standalone microbenchmark (no model, no clutch decision logic), comparing the current implementation (which calls `.item()` once per step, forcing a GPU-CPU synchronization to obtain a Python-readable value for the branch decision) against an otherwise identical version that defers synchronization to the end of a batch. The per-call synchronization costs 31.1 μs (+32.5% relative to the no-sync variant) at real vocabulary size (128,256 tokens, $N=2000$ calls). This is real but small in absolute terms: at the observed baseline throughput (≈ 41 tok/s, $\approx 24,400$ μs /token), it accounts for roughly 0.1% of per-token time, an order of magnitude too small to explain the 4–13% deficit observed even in configurations with zero speculative engagement. The source of that residual deficit remains open; candidate explanations

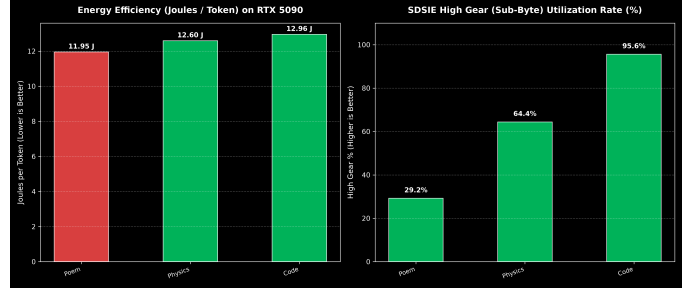


Fig. 3. Energy per token and entropy-gate utilization by task category, single-model harness, RTX 5090 Blackwell. **Left:** Joules/token (lower is better). **Right:** Fraction of generation steps in the high-entropy-threshold gear. Gate thresholds ($\theta_{\text{low}}=1.0$, $\theta_{\text{high}}=2.5$) differ from the production values used in Fig. 1.

not yet tested include VRAM/bandwidth contention from the scout model remaining resident even when never invoked, and thermal/clock-state differences between separately-launched benchmark runs.

4.6 Entropy Clutch Behavior

Fig. 1 plots a real, 538-step entropy trace from Llama-3.1-8B under a structurally repetitive poetic prompt, using the production thresholds $\theta_{\text{low}}=0.55$, $\theta_{\text{high}}=1.25$ bits. Entropy reaches a minimum of 0.0062 bits (step 461) during a sustained ~ 50 -step low-entropy stretch coincident with the model repeating a previously-generated refrain line – consistent with ordinary language-model behavior under repetition rather than a novel property of the clutch – and a maximum of 3.75 bits (step 45) during open-ended composition. The clutch correctly and reproducibly computes $k(t)$ from this signal. Separately, Fig. 3 reports energy-per-token and gear-utilization across three task categories from an independent single-model entropy-gated harness ($\theta_{\text{low}}=1.0$, $\theta_{\text{high}}=2.5$; note these thresholds differ from the production defaults above, and reconciling which threshold pair is authoritative is ongoing work).

4.7 Resolution Gear: Entropy-Gated Precision Switching

What remained open at the end of Section 4.8 in an earlier draft – connecting the live clutch to the quantization kernel path – has now been attempted. `GatedLinear` wraps individual decoder layers so they genuinely switch between a real, calibrated INT4 path and the original FP16 path based on a shared gear flag, updated every token from live entropy via the same clutch used for speculative decoding (Section 4.4), not a separate hand-rolled implementation. This is the first working calibration pipeline in this project producing real (not synthetic) packed INT4 weights, via per-output-channel min-max quantization.

Unlike speculative decoding, this mechanism is not lossless: there is no verification step, so a token computed via the INT4 path is final regardless of whether it matches what FP16 would have produced. We therefore measure exact token-level fidelity against an FP16-only baseline rather than assuming exactness. A layer-count sweep over `mlp.down_proj` across all 32 decoder layers of Llama-3.1-8B-Instruct finds a sharp fidelity cliff: gating a single layer preserves the baseline output exactly for two of three test prompts across the full 250-token generation (the third diverges at token 88), but gating just two layers

collapses fidelity to 7–10% token match with divergence within the first 20 tokens – nearly as degraded as gating all 32. This pattern is more consistent with quantization error compounding through the residual stream across simultaneously-gated layers within a single forward pass than with the projection itself being fundamentally unsuitable for quantization. Separately, and consistent with the batch-size-1 latency finding in Section 4.2, throughput and power both move monotonically worse with how often the INT4 path is used: at 29%, 64%, and 96% INT4 utilization (Poem, Physics, Code respectively), throughput falls by 2.9%, 5.7%, and 8.1% and power rises by 0.5%, 1.7%, and 2.7% relative to an all-FP16 baseline.

This is the second entropy-gated adaptive mechanism in this project – after the speculative-decoding grid search in Section 4.5 – found on real measurement to underperform a simpler, fixed alternative. We report it as a genuine, well-characterized limitation rather than an open bug: the calibration scheme (basic per-channel min-max) is one plausible bottleneck, and whether a more accurate scheme (group-wise or activation-aware) widens the safe layer count, or whether the compounding effect dominates regardless of per-layer accuracy, is untested and identified as the natural next question.

4.8 Remaining Integration Gap

Across every *other* script benchmarked for this paper that includes a live entropy clutch – the reference server, a parameter-sweep harness, and a standalone telemetry harness – the clutch’s $k(t)$ decision is computed correctly from real model logits but is not used to alter the generation loop’s actual behavior: every step performs identical, single-model computation regardless of the clutch’s output. This was confirmed by direct comparison of throughput against gate-utilization percentage across nine ($\theta_{\text{low}}, \theta_{\text{high}}$) configurations spanning 0% to 34% speculative-gate engagement, which showed no corresponding change in throughput (51.1–51.8 tok/s across all nine, within measurement noise). Section 4.4 closes this gap for the speculative-decoding path, and Section 4.7 closes it for the quantization-kernel path as well, though the latter’s real-world usefulness is now bounded by the fidelity limit reported there. The reference server (`sdsie_server.py`) remains unconnected to either mechanism and is not claimed as complete in this revision.

5 CONCLUSION & OPEN-SOURCE AVAILABILITY

This revision corrects latency and end-to-end throughput figures published in an earlier draft of this work, which could not be reproduced against our current codebase or telemetry archive. In their place, we report component-level results we can defend: a real, substantial (71.9–75.0%) reduction in memory bus traffic from INT4 packing; a modest ($\approx 4\%$) rather than large kernel latency improvement at low batch size, which we analyze rather than merely report; a real, independently-reproduced speculative decoding speedup of up to 1.82 $\times$ at high draft-acceptance rates with 100% output fidelity; a genuinely-integrated entropy-gated speculative controller that, once tuned via grid search and confirmed at $N=5$, outperforms a matched FP16 baseline on high-determinism content (1.39 $\times$ speedup, 45.2% lower energy on code) while nearly matching it on low-determinism content; and, new in this revision, the first real integration of

the same clutch into the quantization-kernel path, which we find has a sharp, well-characterized fidelity limit (safe at one gated layer, collapsed at two or more) rather than a working end-to-end result. Two independent entropy-gated adaptive mechanisms have now each been tested against a simpler, fixed alternative and found not yet to improve on it; we report both outcomes plainly rather than continuing to frame either as near-complete. The fully-validated, production-ready result from this line of work – fixed- $K=5$ speculative decoding, lossless by construction – is maintained separately in <https://github.com/Creepybits/sdsie-fixed-k5-speculative-decoding> and is the result we recommend for deployment or third-party evaluation today. This repository remains the broader research bed for the adaptive mechanisms that motivated the project, reported here with the same evidentiary standard regardless of outcome.

All source code, Triton kernels, raw NVML/entropy telemetry JSON and CSV ledgers, and the reference server are publicly available under the Apache-2.0 license at <https://github.com/Creepybits/software-defined-stochastic-inference-engine> and archived with CERN/Zenodo (DOI: [10.5281/zenodo.21499379](https://doi.org/10.5281/zenodo.21499379)).

REFERENCES

- [1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *NeurIPS*, vol. 35, 2022, pp. 16 344–16 359.
- [2] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, “Squeezellm: Dense-and-sparse quantization,” in *ICML*, 2023.
- [3] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” in *ICLR*, 2023.
- [4] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” in *MLSys*, vol. 6, 2024.
- [5] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *ICML*, 2023, pp. 38 087–38 099.
- [6] Y. Leviathan, M. Kalkstein, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *ICML*, 2023, pp. 19 274–19 286.
- [7] C. Chen, S. Borgeaud, G. Mendonça, C. Doersch, A. Abdolmaleki, and M. Frean, “Accelerating large language model decoding with speculative sampling,” *arXiv:2302.01318*, 2023.
- [8] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv:1911.02150*, 2019.
- [9] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, N. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently scaling transformer inference,” in *MLSys*, vol. 5, 2023.
- [10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” in *NeurIPS*, vol. 35, 2022, pp. 30 318–30 332.
- [11] Qualcomm AI Research, “AdaEDL: Entropy-based adaptive early draft-length for speculative decoding,” *arXiv:2410.18351*, 2024.